16 EXCEPTION HANDLING • THEORY

03 standard and custom exception classes

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

03 — Standard Exception Hierarchy (`<stdexcept>`) and Custom Exceptions

1. The Standard `<exception>` and `<stdexcept>` Hierarchy

All standard C++ exceptions inherit from the base class `std::exception` (`#include <exception>`), which provides a virtual member function:

CPP

```cpp
virtual const char* what() const noexcept;
```

Major Standard Exception Classes

1. **`std::logic_error`** (`<stdexcept>`): Errors detectable before execution by validating preconditions:
   - `std::invalid_argument`
   - `std::out_of_range` (thrown by `std::vector::at()` and `std::string::at()`)
   - `std::domain_error`
   - `std::length_error`
2. **`std::runtime_error`** (`<stdexcept>`): Errors detectable only at runtime:
   - `std::overflow_error`
   - `std::underflow_error`
   - `std::range_error`
3. **`std::bad_alloc`** (`<new>`): Thrown by `new` when heap allocation fails.
4. **`std::bad_cast`** (`<typeinfo>`): Thrown by `dynamic_cast<Derived&>(base_ref)` when a reference downcast fails.

2. Golden Rule: Throw by Value, Catch by `const` Reference

CPP

```cpp
try {
    throw runtime_error("Insufficient balance"); // Throw by value
}
catch (const exception& e) {                     // Catch by const reference
    cout << e.what() << endl;
}
```

Why catch by `const exception&`?

1. Avoids **object slicing** (preserves the derived class's overridden `what()` virtual function).
2. Avoids copying the exception object.
3. Avoids heap memory management issues that would occur if throwing pointers (`throw new ...`).

---

3. Writing a Simple Classroom Custom Exception Class

CPP

```cpp
class negative_amount_error {
private:
    int bad_amount;

public:
    negative_amount_error(int amt) {
        bad_amount = amt;
    }

    void show_error() const {
        cout << "Invalid negative amount: " << bad_amount << endl;
    }
};
```

Or, when integrating with the C++ standard library, inherit publicly from `std::exception` and override `const char* what() const noexcept override`.